

1. Install Java (Required)

Spark + Scala run on JVM.

Install Java 11

Download **OpenJDK 11** from:

- Adoptium Temurin JDK:
<https://adoptium.net>

Choose:

Windows x64

JDK 11

.msi installer

Install it normally.

3 Update JAVA_HOME

After installation:

1. Open **Environment Variables**
2. Set:

JAVA_HOME = C:\Program Files\Eclipse Adoptium\jdk-11.x.x

3. Update **Path** to include:

%JAVA_HOME%\bin

2 Install Scala

Download:

<https://www.scala-lang.org/download/>

Check installation:

```
scala -version
```

Example output

```
Scala code runner version 2.12
```

Also verify compiler:

```
scalac -version
```

3 Install IntelliJ IDEA (Recommended)

Download:

<https://www.jetbrains.com/idea/>

Install plugin

Scala Plugin

Steps:

New Project

→ Scala

→ sbt project

4 Install Spark

Download

<https://spark.apache.org/downloads.html>

Choose

Spark 3.x

Scala 2.12

Extract

```
tar -xvf spark-3.x-bin-hadoop3.tgz
```

Set environment variable

```
SPARK_HOME= C:\spark\spark-3.5.8-bin-hadoop3-scala2.13
```

```
PATH= %SPARK_HOME%\bin
```

Test:

```
spark-shell
```

You should see

```
Scala version 2.12
```

```
Spark context available
```

For java jdk version 16 above

add VM options in IntelliJ run modify options

```
--add-exports java.base/sun.nio.ch=ALL-UNNAMED --add-opens  
java.base/sun.nio.ch=ALL-UNNAMED
```

Build.sbt

```
name := "spark"  
  
version := "1.0"  
  
scalaVersion := "2.12.18"  
  
libraryDependencies ++= Seq(  
  "org.apache.spark" %% "spark-core" % "3.5.1",  
  "org.apache.spark" %% "spark-sql" % "3.5.1"  
)
```

Initial spark code

```
import org.apache.spark.sql.SparkSession

object sparksession {

  def main(args: Array[String]): Unit = {

    val spark = SparkSession.builder()
      .appName("ScalaSparkPipeline")
      .master("local[*]")
      .getOrCreate()

    println("Spark Started")

    spark.stop()
  }
}
```

1. Definition of Scala

Scala is a **statically typed programming language** that runs on the **JVM (Java Virtual Machine)** and combines **object-oriented programming** with **functional programming**.

It is designed to support **high-performance, distributed, and concurrent systems**, which makes it widely used in **big data processing frameworks such as Apache Spark, Kafka Streams, Flink, and Akka**.

In data engineering systems, Scala is commonly used to build:

- ETL pipelines
- Distributed data processing jobs
- Streaming data pipelines
- Data transformation logic
- High-performance backend services for data platforms

Scala compiles to **Java bytecode**, allowing it to run on any system that supports the JVM.

2. Why Scala is Important in Data Engineering

Data engineering systems must process **large volumes of distributed data** efficiently and reliably.

Scala provides several characteristics that are well suited for this environment.

1. JVM Performance

Scala runs on the JVM, which provides:

- optimized memory management
- efficient garbage collection
- mature concurrency support

This allows large-scale data processing systems to run reliably.

2. Functional Programming Support

Many distributed systems rely on **functional transformations of data**.

Examples in Spark include:

```
map  
filter  
flatMap  
reduce  
groupBy
```

These operations depend heavily on functional programming concepts such as:

- higher-order functions
- immutability

- pure functions

Scala natively supports these constructs.

3. Immutability

Distributed systems often process data across multiple nodes.

Mutable state can cause problems such as:

- race conditions
- inconsistent results
- synchronization issues

Scala encourages immutable variables using `val`.

Example:

```
val totalRecords = 1000
```

Once assigned, the value cannot change.

This improves reliability in distributed computations.

4. Concise Syntax

Scala code is typically more concise than Java.

Example (Java style):

```
for(int i = 0; i < numbers.length; i++){  
    System.out.println(numbers[i]);  
}
```

Scala equivalent:

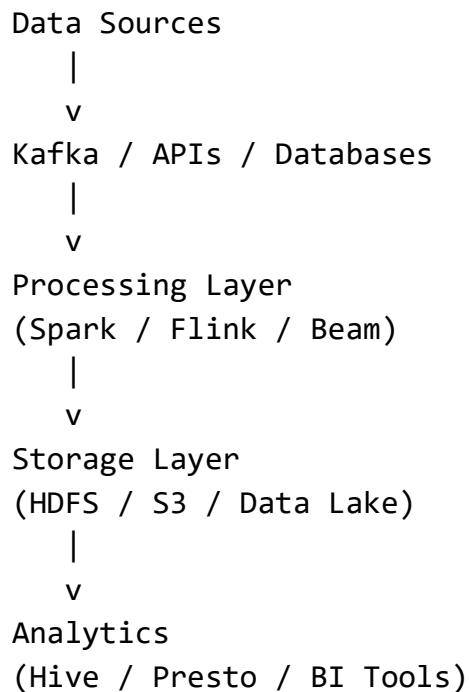
```
numbers.foreach(println)
```

This makes transformation logic easier to read and maintain.

3. Scala in Data Engineering Architecture

In typical data platforms, Scala is used primarily in the **processing layer**.

Example pipeline:



Scala is often used to implement logic in:

- Spark batch jobs
- streaming pipelines
- data transformation services

Spark itself is implemented in Scala, which is why the **native Spark API is written in Scala**.

4. Scala Program Structure

A Scala application normally starts with an **object** containing an entry point.

Example:

```
object MyProgram {  
  
    def main(args: Array[String]): Unit = {  
  
        println("Scala Application")  
  
    }  
  
}
```

Another common approach uses App:

```
object MyProgram extends App {  
  
    println("Scala Application")  
  
}
```

5. Variables in Scala

Scala supports two types of variables.

Keyword	Description
val	Immutable variable
var	Mutable variable

Immutable Variable

```
val company = "EnhanceIT"
```

Value cannot be changed.

```
company = "OtherCompany"
```

This will produce an error.

Immutability is preferred in distributed systems.

Mutable Variable

```
var count = 10  
count = 20
```

Value can change.

Mutable variables should be minimized in distributed programs.

LAB 1 – Variables

Create a Scala program.

File name: VariablesLab.scala

```
object VariablesLab extends App {
```

```
    val company = "DataPlatform"  
    var engineers = 5
```

```
    println(company)  
    println(engineers)
```

```
    engineers = engineers + 3
```

```
    println(engineers)
```

```
}
```

Expected Output

DataPlatform

5

8

Exercise 1

Create variables for:

- employeeName
- department
- salary

Print them.

Solution

```
object Exercise1 extends App {  
  
    val employeeName = "Alice"  
    val department = "Engineering"  
    var salary = 90000  
  
    println(employeeName)  
    println(department)  
    println(salary)  
  
}
```

6. Data Types

Scala is **strongly typed**, meaning every variable has a defined data type.

Common primitive types include:

Type	Example
Int	10
Double	10.5
Boolean	true
String	"Scala"

Long	100000L
------	---------

Example:

```
val age: Int = 30
val salary: Double = 90000.50
val isActive: Boolean = true
```

Scala also supports **type inference**.

Example:

```
val age = 30
```

The compiler automatically determines that age is an integer.

LAB 2 – Data Types

File: DataTypesLab.scala

```
object DataTypesLab extends App {
```

```
    val name: String = "Bob"
    val age: Int = 28
    val salary: Double = 72000.75
    val active: Boolean = true
```

```
    println(name)
    println(age)
    println(salary)
    println(active)
```

```
}
```

Exercise 2

Create variables for:

- product
- price
- quantity

Calculate total price.

Solution

```
object Exercise2 extends App {  
  
    val product = "Laptop"  
    val price = 1000  
    val quantity = 3  
  
    val total = price * quantity  
  
    println(total)  
  
}
```

7. String Operations

Strings are immutable in Scala.

Example:

```
val name = "Data"  
val role = name + " Engineer"
```

This produces a new string object.

String Interpolation

Scala provides convenient string formatting.

Example:

```
val name = "Alice"
val salary = 90000

println(s"$name earns $salary")

Another example with formatting:

println(f"Salary = $salary%.2f")
```

8. Conditional Statements

Scala uses similar conditional structures as other languages.

Example:

```
val salary = 80000

if(salary > 75000) {
  println("High Salary")
}
else {
  println("Regular Salary")
}
```

If Expression

In Scala, `if` can return a value.

Example:

```
val result =
  if(age > 18) "Adult"
  else "Minor"
```

The variable `result` will contain the output of the condition.

LAB 3 – Conditions

File: `SalaryCheck.scala`

```
object SalaryCheck extends App {  
  
    val salary = 85000  
  
    if(salary > 80000) {  
        println("Senior Employee")  
    }  
    else {  
        println("Junior Employee")  
    }  
  
}
```

Exercise 3

Write a program that checks whether a number is even or odd.

Solution

```
object EvenOdd extends App {  
  
    val num = 10  
  
    if(num % 2 == 0)  
        println("Even")  
    else  
        println("Odd")  
  
}
```

9. Loops

Scala supports iteration using **for loops**.

For Loop

Example:

```
for(i <- 1 to 5) {  
  println(i)  
}
```

Output:

```
1  
2  
3  
4  
5
```

LAB 4 – Loops

File: LoopLab.scala

```
object LoopLab extends App {
```

```
  for(i <- 1 to 10) {  
    println(i)  
  }
```

```
}
```

Exercise 4

Print numbers from 1 to 20 that are divisible by 3.

Solution

```
object DivisibleBy3 extends App {  
  
  for(i <- 1 to 20) {  
    if(i % 3 == 0)  
      println(i)  
  }  
  
}
```

Iterating Collections

Example:

```
val numbers = List(10,20,30)  
  
for(n <- numbers) {  
  println(n)  
}
```

10. Functions

Functions are fundamental in Scala. Functions allow reusable logic.

Example:

```
def add(a: Int, b: Int): Int = {  
  a + b  
}
```

Calling the function:

```
add(2,3)
```

Output:

5

LAB 5 – Functions

File: FunctionLab.scala

```
object FunctionLab extends App {  
  
    def multiply(a:Int, b:Int):Int = {  
        a * b  
    }  
  
    val result = multiply(5,6)  
  
    println(result)  
  
}
```

Exercise 5

Create a function that calculates square of a number.

Solution

```
object SquareFunction extends App {  
  
    def square(x:Int):Int = {  
        x * x  
    }  
  
    println(square(4))  
  
}
```

11. Types of Functions

Pure Functions

A pure function produces the same output for the same input and does not modify external state.

Example:

```
def square(x: Int): Int = {  
    x * x  
}
```

Pure functions are ideal for distributed systems because they are predictable.

Impure Functions

Impure functions rely on external state.

Example:

```
var counter = 0  
  
def increment(): Int = {  
    counter += 1  
    counter  
}
```

This can cause issues in distributed systems.

12. Anonymous Functions

Anonymous functions do not have names.

Example:

```
val multiply = (x: Int) => x * 2
```

These are heavily used in Spark transformations.

Example:

```
List(1,2,3).map(x => x * 2)
```

LAB 6 – Anonymous Functions

File: LambdaLab.scala

```
object LambdaLab extends App {  
  
    val numbers = List(1,2,3,4,5)  
  
    val doubled = numbers.map(x => x * 2)  
  
    println(doubled)  
  
}
```

Exercise 6

Filter numbers greater than 10.

Solution

```
object FilterExercise extends App {  
  
    val nums = List(5,12,18,3,9)  
  
    val result = nums.filter(x => x > 10)  
  
    println(result)
```

```
}
```

13. Higher Order Functions

A higher-order function either:

- accepts another function as a parameter
- returns a function as a result

Example:

```
def applyFunc(f: Int => Int, value: Int): Int = {  
  f(value)  
}
```

```
applyFunc(x => x * 2, 10)
```

14. Scala Collections

Collections are central to Scala programming and form the conceptual foundation for Spark transformations.

Lists

Lists are immutable sequences.

Example:

```
val numbers = List(1,2,3,4)
```

Operations:

```
numbers.map(x => x * 2)
numbers.filter(x => x > 2)
numbers.reduce(_ + _)
```

LAB 7 – List Operations

```
File: ListOperations.scala
object ListOperations extends App {

    val numbers = List(1,2,3,4,5)

    val doubled = numbers.map(x => x*2)

    val even = numbers.filter(x => x%2==0)

    println(doubled)
    println(even)

}
```

Exercise 7

Find sum of numbers in list.

Solution

```
object SumList extends App {

    val nums = List(10,20,30,40)

    val total = nums.reduce(_ + _)

    println(total)

}
```

Maps

Maps store key-value pairs.

Example:

```
val employees = Map(  
  1 -> "Alice",  
  2 -> "Bob",  
  3 -> "Charlie"  
)
```

Access value:

```
employees(1)
```

Tuples

Tuples can store multiple values of different types.

Example:

```
val record = (1, "Alice", 90000)
```

Access fields:

```
record._1  
record._2  
record._3
```

Tuples are commonly used in key-value operations.

LAB 8 – Tuple Example

```
object TupleLab extends App {
```

```
  val record = (101,"Alice","Engineering")
```

```
println(record._1)
println(record._2)
println(record._3)

}
```

15. Case Classes

Case classes are specialized classes designed for immutable data structures.

Example:

```
case class Employee(name: String, age: Int)
```

Create instance:

```
val emp = Employee("John", 30)
```

Case classes automatically provide:

- equality comparison
- string representation
- serialization support

This makes them ideal for representing structured data in Spark datasets.

Case classes automatically support serialization and pattern matching, making them suitable for distributed systems like Spark

LAB 9 – Case Class

```
case class Employee(id:Int,name:String,salary:Int)
```

```
object CaseClassLab extends App {
```

```
    val employees = List(
        Employee(1,"Alice",90000),
        Employee(2,"Bob",75000),
```

```
        Employee(3, "Charlie", 60000)
    )

    val highSalary =
        employees.filter(e => e.salary > 80000)

    println(highSalary)
}
```

16. Traits

Traits are similar to interfaces but can include implementation.

Example:

```
trait Logger {

    def log(msg: String): Unit = {
        println(msg)
    }

}
```

Traits are used for:

- logging
- configuration logic
- reusable functionality

17. Serialization

In distributed systems, objects must be transferred between machines.

Serialization converts an object into a binary representation that can be sent across the network.

Example:

```
case class LogEvent(level: String, message: String)
```

Spark automatically serializes such objects for execution across cluster nodes.

18. Example Data Engineering Transformation

Input dataset:

```
1,Alice,Engineering,90000
2,Bob,Engineering,85000
3,Charlie,HR,60000
```

Scala transformation:

```
case class Employee(id:Int,name:String,dept:String,salary:Int)
```

```
val data = List(
  "1,Alice,Engineering,90000",
  "2,Bob,Engineering,85000",
  "3,Charlie,HR,60000"
)
```

```
val employees =
  data.map{ line =>
```

```
    val cols = line.split(",")
```

```
    Employee(
      cols(0).toInt,
      cols(1),
      cols(2),
```

```
        cols(3).toInt  
    )  
  
}
```

Filter:

```
employees.filter(_.salary > 80000)
```

This style of transformation closely resembles Spark data processing logic.

End-of-Session Practice

1. Find average salary from employee list
2. Group employees by department
3. Count employees in each department
4. Find highest salary employee